### The "Must-Haves" for this Project

If you want an SDE intern role, this project needs to demonstrate these four core competencies:

1. **The Actual Use Case:** Routing what? Where? Abstract math is less impressive than solving a real-world problem.
2. **Algorithmic Complexity handled well:** You mentioned $O(N^2)$ and ALNS. That is great, but frame it around *efficiency*, not just the fact that it exists.
3. **Concrete Metrics (The "X-Y-Z" Formula):** You achieved X, by doing Y, resulting in Z. We need placeholder numbers for you to fill in.
4. **Standard SWE Patterns:** Mentioning Strategy/Factory patterns is good. You need to keep that, but tie it to the business logic.

### The Step-by-Step Rewrite

We are going to condense your four bullets into three high-impact bullets.

### Bullet 1: The Core Application & Algorithm (The "What and How")

- **The Flaw in Original:** You focused entirely on "serverless" and "infinite horizontal scaling".
- **The Fix:** Ground it in the actual logistics problem. Show me the scale of the data and the logic.
- **Draft to Use:**

> Developed a hybrid decision engine for real-time e-commerce delivery routing, implementing Strategy and Factory patterns to dynamically switch between Google OR-Tools and custom ALNS heuristics.

### Bullet 2: Performance & Data Handling (The "Engineering Muscle")

- **The Flaw in Original:** You said "massive geocoding" and "preventing timeouts", which sounds like you were just fighting fires rather than engineering a smooth system.
- **The Fix:** Quantify the data and highlight the asynchronous pipeline as a deliberate, performant choice.
- **Draft to Use:**

> Engineered an asynchronous processing pipeline (FastAPI, Redis) to decouple $O(N^2)$ travel-time matrix computations, successfully processing high-density routing datasets across complex subzones (e.g., Delhi NCR, Sector 4, Greater Kailash 2) while maintaining a sub-[X]-second response time.

### Bullet 3: The User Interface & Impact (The "So What?")

- **The Flaw in Original:** It reads like a feature list ("responsive, polling-based... granular KPIs").

- **Draft to Use:**

  > Built a React frontend with polling-based state management to visualize live algorithmic execution, enabling real-time monitoring of chronological simulation events and tracking reductions in overall fleet costs by [X]%.

Why this format works better:

- **It's grounded:** By mentioning e-commerce and specific high-density subzones like Greater Kailash 2 and Sector 4, you instantly prove this wasn't just a toy project—it handled real, messy, geographical data.
- **It proves SWE skills:** You kept the design patterns (Strategy/Factory) and the asynchronous queuing (Redis/FastAPI), which shows you know how to build maintainable backend systems.
- **It removes the hyperbole:** We stripped out "infinite horizontal scaling" and replaced it with tangible system behavior (handling $O(N^2)$ calculations without blocking the main thread).